Implementasi Particle Filter pada Sistem IoT Berbasis ESP32 dan MPU6050

Author(s): Sri Kusmiyati
Teacher: Muhammad Qomaruz Zaman, S.T., M.T., Ph.D.
Class name: Sistem Berbasis Mikroprosesor
YouTube link: `https://youtu.be/MSb_ykKuS6w`
GitHub link: `https://github.com/SKusmiyati/iot-particle-filter`

1. Deskripsi Proyek

Penelitian ini membahas implementasi metode Particle Filter untuk mereduksi noise pada data sensor inersial MPU6050 dalam sistem berbasis Internet of Things (IoT). Particle Filter merupakan metode estimasi berbasis probabilistik yang efektif untuk sistem non-linear dan non-Gaussian [1]. Sistem yang dikembangkan menggunakan ESP32 sebagai perangkat akuisisi data dan Raspberry Pi sebagai server pemrosesan. Pendekatan ini sejalan dengan konsep sistem IoT modern yang mengintegrasikan perangkat edge dan cloud dalam pengolahan data [2].

Pada skenario cloud filtering, data mentah dikirim ke server untuk diproses menggunakan Particle Filter, sedangkan pada skenario edge filtering, proses filtering dilakukan langsung pada ESP32 sebelum data dikirim. Hasil pengujian menunjukkan bahwa Particle Filter mampu mereduksi noise secara efektif pada kedua skenario. Namun, edge filtering menghasilkan sinyal yang lebih stabil dan efisien dalam penggunaan bandwidth, sementara cloud filtering menawarkan fleksibilitas komputasi yang lebih tinggi.

Selain itu, waktu pemrosesan pada skenario cloud menunjukkan bahwa sistem mampu bekerja secara real-time dengan rata-rata waktu kurang dari 1 ms per sampel. Hasil penelitian ini menunjukkan bahwa pemilihan lokasi pemrosesan memiliki pengaruh signifikan terhadap performa sistem, sehingga perlu disesuaikan dengan kebutuhan aplikasi.

2. Metodologi

Penelitian ini menggunakan pendekatan eksperimental untuk membandingkan performa metode filtering pada dua skenario, yaitu cloud filtering dan edge filtering. Sistem dirancang untuk membaca data percepatan dari sensor MPU6050, kemudian dilakukan proses filtering menggunakan Particle Filter pada dua lokasi yang berbeda. Tujuan utama dari desain ini adalah untuk mengevaluasi kualitas sinyal hasil filtering, waktu pemrosesan (processing time) dan efisiensi sistem.

2.1 Arsitektur Sistem

Arsitektur sistem pada penelitian ini dirancang untuk mengintegrasikan proses akuisisi data, transmisi, serta pengolahan sinyal dalam suatu sistem berbasis Internet of Things (IoT). Sistem terdiri dari sensor MPU6050 sebagai sumber data, ESP32 sebagai perangkat akuisisi dan komunikasi, serta Raspberry Pi sebagai server pemrosesan.

Perancangan arsitektur ini bertujuan untuk mendukung implementasi dua pendekatan filtering, yaitu cloud filtering dan edge filtering, sehingga memungkinkan analisis perbandingan performa berdasarkan lokasi pemrosesan data. Dengan struktur ini, aliran data dapat diamati secara menyeluruh mulai dari sensor hingga tahap analisis. Sistem terdiri dari tiga komponen utama:

- Sensor MPU6050 : Mengukur percepatan pada sumbu tertentu.
- ESP32 (Node Sensor / Edge Device) : Membaca data sensor, mengirim data melalui WiFi, dan pada skenario B melakukan filtering.
- Raspberry Pi (Server) : Menerima data melalui UDP, pada skenario A melakukan filtering, dan melakukan logging dan visualisasi.

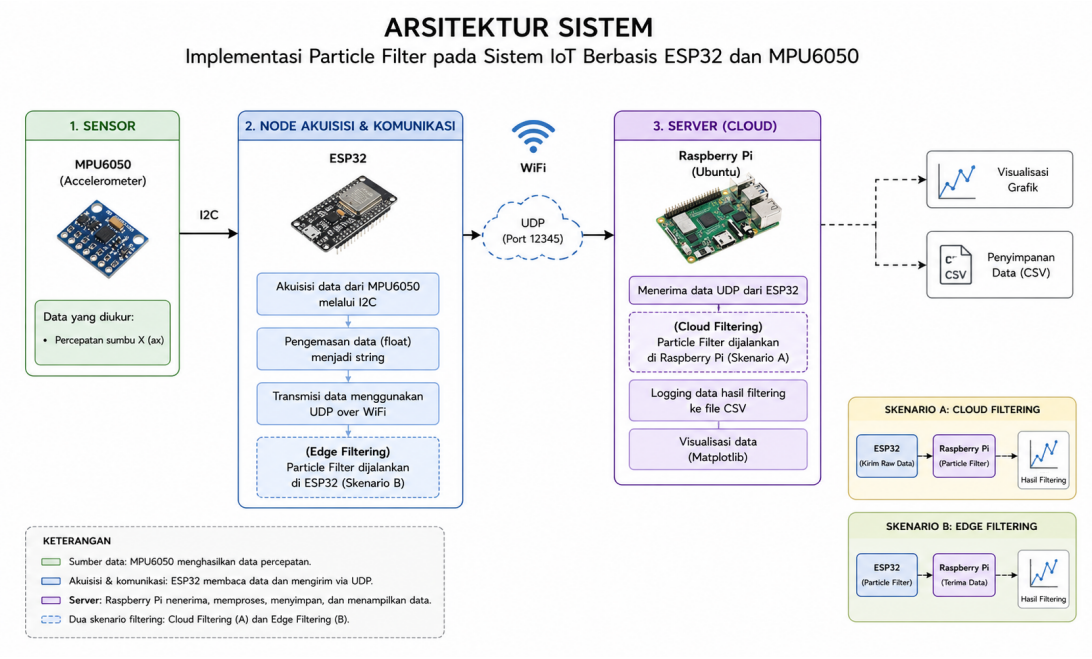

Figure 1: Arsitektur IMU.

2.2 Perangkat Keras

Perangkat keras yang digunakan dalam penelitian ini terdiri dari beberapa komponen utama yang saling terintegrasi dalam membentuk sistem Internet of Things (IoT). ESP32 merupakan mikrokontroler yang banyak digunakan dalam sistem IoT karena memiliki fitur komunikasi WiFi dan kemampuan pemrosesan yang cukup baik [3]. Sensor MPU6050 digunakan sebagai sumber data percepatan dan kecepatan sudut yang umum digunakan dalam sistem embedded [4]. Data dari sensor ini dibaca oleh ESP32 melalui komunikasi I2C menggunakan kabel jumper sebagai media penghubung antar perangkat.

Selanjutnya, Raspberry Pi digunakan sebagai server yang berfungsi untuk menerima data yang dikirim oleh ESP32 melalui jaringan WiFi. Selain itu, Raspberry Pi juga bertanggung jawab dalam melakukan pemrosesan data, khususnya pada skenario cloud filtering, serta menyimpan dan menampilkan hasil pengolahan data. Seluruh sistem terhubung melalui jaringan WiFi yang memungkinkan komunikasi data secara real-time tanpa menggunakan kabel fisik.

2.3 Perangkat Lunak

Perangkat lunak yang digunakan dalam penelitian ini mendukung proses pengembangan sistem mulai dari akuisisi data hingga visualisasi hasil. Arduino IDE digunakan sebagai platform utama untuk melakukan pemrograman pada ESP32, termasuk pengaturan komunikasi WiFi dan pengiriman data menggunakan protokol UDP.

Pada sisi server, bahasa pemrograman Python digunakan untuk menerima data, melakukan pemrosesan menggunakan algoritma Particle Filter, serta menampilkan hasil dalam bentuk visualisasi grafik. Dalam implementasinya, beberapa library digunakan untuk mendukung fungsi sistem, di antaranya library WiFi dan WiFiUDP untuk komunikasi jaringan pada ESP32, library Wire untuk komunikasi I2C dengan sensor MPU6050, serta library NumPy dan Matplotlib pada Python untuk pengolahan data numerik dan visualisasi hasil secara grafis.

2.4 Konfigurasi Sistem

Konfigurasi sistem pada penelitian ini mencakup pengaturan komunikasi antara ESP32 dan Raspberry Pi, serta parameter algoritma Particle Filter yang digunakan dalam proses pengolahan data. Konfigurasi ini meliputi pengaturan alamat IP, port komunikasi UDP, serta jumlah partikel yang digunakan dalam filtering.

Pengaturan tersebut dirancang untuk memastikan kestabilan komunikasi data dan performa sistem secara keseluruhan, sehingga proses pengiriman dan pengolahan data dapat dilakukan secara real-time pada kedua skenario yang diujikan.

Konfigurasi ini menjadi dasar dalam pelaksanaan pengujian sistem dan analisis hasil yang diperoleh pada tahap selanjutnya.

- Koneksi MPU6050 ke ESP32 menggunakan protokol I2C:

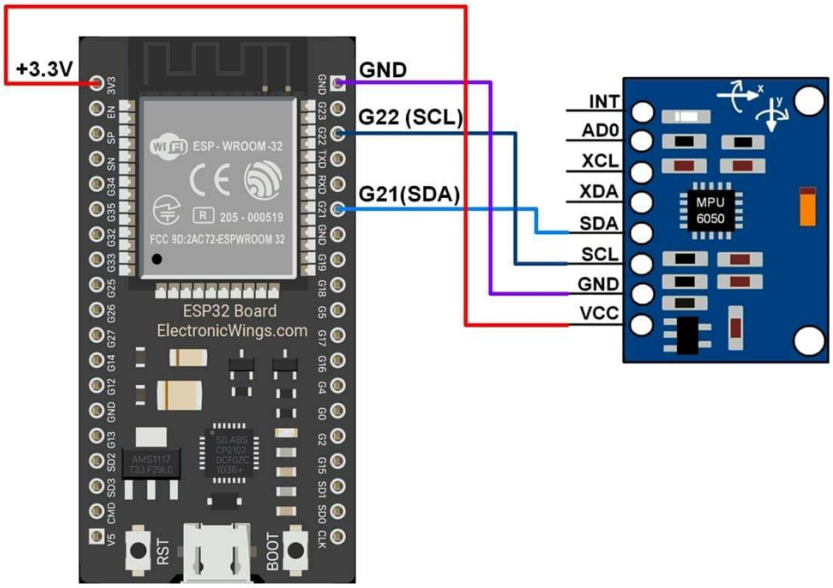

Figure 2: Konfigurasi MPU6050 ke ESP32 menggunakan protokol I2C.

- Data dikirim menggunakan protokol UDP dengan format:

  Skenario A: "ax"

  Skenario B: "ax,filtered"

2.5 Implementasi Particle Filter

Implementasi Particle Filter pada penelitian ini dilakukan untuk mereduksi noise pada data percepatan yang diperoleh dari sensor MPU6050. Algoritma Particle Filter dipilih karena kemampuannya dalam menangani sistem dengan karakteristik non-linear dan noise yang tidak terdistribusi secara Gaussian [1].

- Inisialisasi

  sejumlah partikel dibangkitkan secara acak untuk merepresentasikan kemungkinan nilai keadaan sistem. Setiap partikel memiliki bobot awal yang sama, yang kemudian akan diperbarui berdasarkan kesesuaian antara prediksi partikel dengan data pengukuran yang diterima dari sensor.

- Prediksi

  setiap partikel diperbarui dengan menambahkan noise acak yang mengikuti distribusi tertentu untuk mensimulasikan dinamika sistem. Selanjutnya, pada tahap update, bobot setiap partikel dihitung berdasarkan selisih antara nilai partikel dan data pengukuran, sehingga partikel yang lebih mendekati nilai pengukuran akan memiliki bobot yang lebih besar.

- Resampling

  untuk menghindari degenerasi partikel, yaitu kondisi di mana hanya beberapa partikel yang memiliki bobot signifikan. Pada tahap ini, partikel dengan bobot tinggi akan lebih sering dipilih, sedangkan partikel dengan bobot rendah akan dieliminasi.

- Estimasi

  nilai akhir diperoleh dengan menghitung rata-rata dari seluruh partikel yang telah diperbarui. Hasil estimasi ini kemudian digunakan sebagai data hasil filtering yang memiliki tingkat noise lebih rendah dibandingkan dengan data mentah.

Implementasi algoritma ini dilakukan pada dua skenario, yaitu pada sisi server (cloud filtering) dan pada sisi perangkat edge (edge filtering). Pada skenario cloud filtering, seluruh proses Particle Filter dijalankan pada Raspberry Pi setelah data diterima melalui UDP. Sedangkan pada skenario edge filtering, proses filtering dilakukan langsung pada ESP32 sebelum data dikirim ke server. Dengan pendekatan ini, dapat dilakukan analisis terhadap pengaruh lokasi pemrosesan terhadap kualitas sinyal, efisiensi sistem, serta performa komputasi secara keseluruhan.

2.6 Skenario Pengujian

Penelitian ini menggunakan dua skenario:

- Skenario A (Cloud Filtering)

  MPU6050 → ESP32 → RAW data → Raspberry Pi → Particle Filter.

  Pada skenario A, proses pengolahan data dilakukan pada sisi server (cloud), yaitu Raspberry Pi. Data mentah yang diperoleh dari sensor MPU6050 dikirimkan oleh ESP32 melalui komunikasi UDP tanpa dilakukan proses filtering terlebih dahulu pada perangkat edge.

  Setelah data diterima oleh Raspberry Pi, proses Particle Filter dijalankan untuk mereduksi noise pada sinyal percepatan. Hasil filtering kemudian digunakan untuk analisis lebih lanjut serta visualisasi data.

  Pendekatan ini memungkinkan penggunaan sumber daya komputasi yang lebih besar pada sisi server, sehingga proses filtering dapat dilakukan dengan lebih fleksibel. Namun, skenario ini memiliki ketergantungan terhadap kualitas jaringan karena seluruh data mentah harus dikirim terlebih dahulu sebelum diproses.

- Skenario B (Edge Filtering)

  MPU6050 → ESP32 → Particle Filter → Filtered data → Raspberry Pi

  Pada skenario B, proses pengolahan data dilakukan langsung pada perangkat edge, yaitu ESP32, sebelum data dikirim ke server. Data percepatan yang diperoleh dari sensor MPU6050 tidak hanya dibaca, tetapi juga langsung diproses menggunakan algoritma Particle Filter pada perangkat tersebut.

  Hasil filtering kemudian dikirim ke Raspberry Pi melalui komunikasi UDP dalam kondisi yang telah diproses, sehingga server hanya berfungsi sebagai penerima data, penyimpan, serta media visualisasi. Dengan demikian, beban komputasi pada sisi server dapat dikurangi secara signifikan.

  Pendekatan ini memungkinkan pengurangan penggunaan bandwidth serta meningkatkan efisiensi sistem karena data yang dikirim telah mengalami proses reduksi noise. Selain itu, filtering yang dilakukan pada sisi edge juga memberikan respon yang lebih cepat terhadap perubahan data, meskipun memerlukan optimasi algoritma agar dapat berjalan pada perangkat dengan keterbatasan sumber daya.

2.7 Analisis Data

Metode analisis data pada penelitian ini dilakukan untuk mengevaluasi performa sistem dalam mereduksi noise serta membandingkan efektivitas dua pendekatan filtering, yaitu cloud filtering dan edge filtering. Data hasil pengujian terlebih dahulu direkam dalam bentuk numerik, kemudian dianalisis menggunakan pendekatan visual dan statistik.

Pendekatan ini memungkinkan evaluasi performa sistem secara komprehensif, khususnya dalam konteks sistem Internet of Things (IoT) yang menuntut pemrosesan data secara real-time dan efisien [5]. Selain itu, penggunaan Particle Filter sebagai metode filtering didasarkan pada kemampuannya dalam menangani sistem non-linear dan noise yang bersifat non-Gaussian [1].

Analisis visual dilakukan dengan menampilkan grafik perbandingan antara sinyal mentah dan sinyal hasil filtering. Melalui visualisasi ini, dapat diamati tingkat fluktuasi sinyal serta kemampuan algoritma Particle Filter dalam mereduksi noise tanpa menghilangkan karakteristik utama sinyal. Selain itu, grafik juga digunakan untuk membandingkan karakteristik sinyal pada kedua skenario, sehingga perbedaan performa dapat diidentifikasi secara langsung.

Selanjutnya, analisis statistik dilakukan untuk memberikan evaluasi kuantitatif terhadap hasil filtering. Parameter yang digunakan meliputi nilai rata-rata dan standar deviasi, yang digunakan untuk mengukur kestabilan sinyal. Nilai standar deviasi yang lebih kecil menunjukkan bahwa sinyal memiliki tingkat noise yang lebih rendah dan tingkat kestabilan yang lebih baik.

Selain analisis statistik dasar, dilakukan pula analisis matematis untuk mengevaluasi performa filtering secara kuantitatif. Parameter yang digunakan meliputi nilai rata-rata, standar deviasi, dan Mean Squared Error (MSE).

Nilai rata-rata dihitung untuk mengetahui kecenderungan nilai sinyal, yang dirumuskan sebagai:

$$\mu = \frac{1}{N}\sum_{i=1}^{N} x_i \tag{1}$$

Sedangkan standar deviasi digunakan untuk mengukur tingkat penyebaran data terhadap nilai rata-rata, yang dirumuskan sebagai:

$$\sigma = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^2} \tag{2}$$

Nilai standar deviasi yang lebih kecil menunjukkan bahwa sinyal memiliki tingkat noise yang lebih rendah dan lebih stabil.

Selain itu, digunakan pula Mean Squared Error (MSE) untuk mengukur perbedaan antara sinyal mentah dan sinyal hasil filtering, yang dirumuskan sebagai:

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(x_i - \hat{x}_i)^2 \tag{3}$$

di mana x_i merupakan data mentah dan $\hat{x}_i$ merupakan hasil estimasi dari Particle Filter.

Penggunaan MSE memberikan gambaran kuantitatif mengenai seberapa baik algoritma filtering dalam mendekati nilai sebenarnya. Nilai MSE yang lebih kecil menunjukkan bahwa hasil filtering semakin mendekati kondisi ideal. Dengan menggunakan parameter-parameter tersebut, evaluasi performa sistem tidak hanya dilakukan secara visual, tetapi juga didukung oleh analisis matematis yang lebih objektif dan terukur.

Selain itu, waktu pemrosesan juga dianalisis untuk mengevaluasi kemampuan sistem dalam bekerja secara real-time. Pengukuran waktu dilakukan pada setiap proses filtering untuk memperoleh rata-rata waktu komputasi per sampel, sehingga dapat diketahui efisiensi sistem pada masing-masing skenario.

Melalui kombinasi analisis visual dan statistik tersebut, diperoleh gambaran yang lebih komprehensif mengenai performa sistem, serta perbedaan karakteristik antara pendekatan cloud dan edge dalam pengolahan data sensor.

3. Hasil dan Pembahasan

Bagian ini menyajikan hasil pengujian sistem yang telah dirancang serta analisis terhadap performa metode Particle Filter dalam mereduksi noise pada data sensor MPU6050. Metode Particle Filter diketahui efektif dalam menangani sistem non-linear dan noise yang tidak terdistribusi secara Gaussian [1]. Pengujian dilakukan dengan dua skenario utama, yaitu cloud filtering dan edge filtering, untuk mengevaluasi pengaruh lokasi pemrosesan terhadap kualitas sinyal dan efisiensi sistem.

Hasil yang diperoleh ditampilkan dalam bentuk visualisasi grafik dan data pengukuran, yang kemudian dianalisis untuk mengidentifikasi karakteristik sinyal sebelum dan sesudah proses filtering. Selain itu, dilakukan pula evaluasi terhadap waktu pemrosesan untuk menilai kemampuan sistem dalam bekerja secara real-time.

Melalui pembahasan ini, diharapkan dapat diperoleh pemahaman yang komprehensif mengenai kelebihan dan keterbatasan masing-masing pendekatan, serta memberikan dasar dalam menentukan metode yang paling sesuai untuk implementasi sistem berbasis Internet of Things.

3.1 Hasil Pengujian Skenario A

Figure 3: Tampilan pada Arduino IDE saat running skenario A

Berdasarkan Gambar di atas, terlihat bahwa implementasi skenario A (cloud filtering) berjalan dengan baik, di mana ESP32 berhasil mengirimkan data mentah dari sensor MPU6050 ke Raspberry Pi melalui komunikasi UDP. Proses pengiriman data yang berlangsung secara kontinu menunjukkan bahwa sistem mampu bekerja secara real-time.

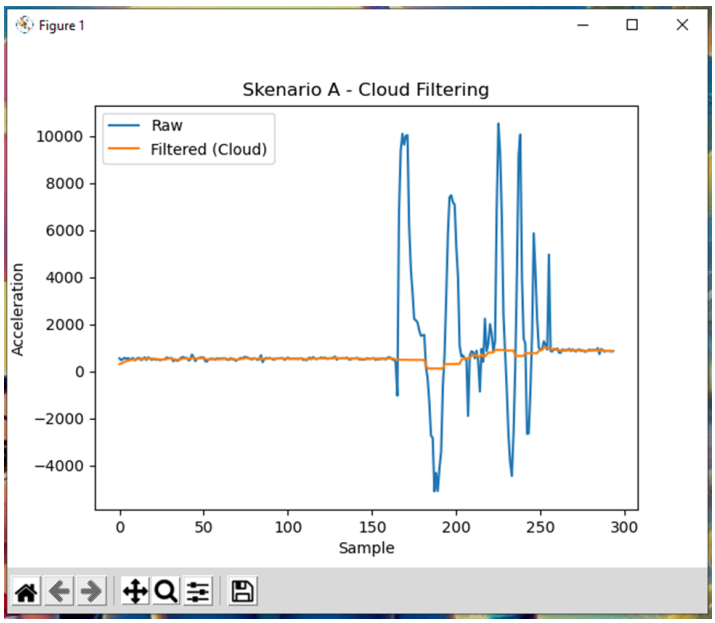

Figure 4: Cloud Filtering

Hasil pada Gambar cloud filtering menunjukkan bahwa sinyal mentah yang memiliki fluktuasi tinggi dapat direduksi secara signifikan setelah diproses menggunakan Particle Filter. Sinyal hasil filtering menjadi lebih halus dan stabil, yang menunjukkan bahwa algoritma mampu mengurangi noise tanpa menghilangkan karakteristik utama sinyal.

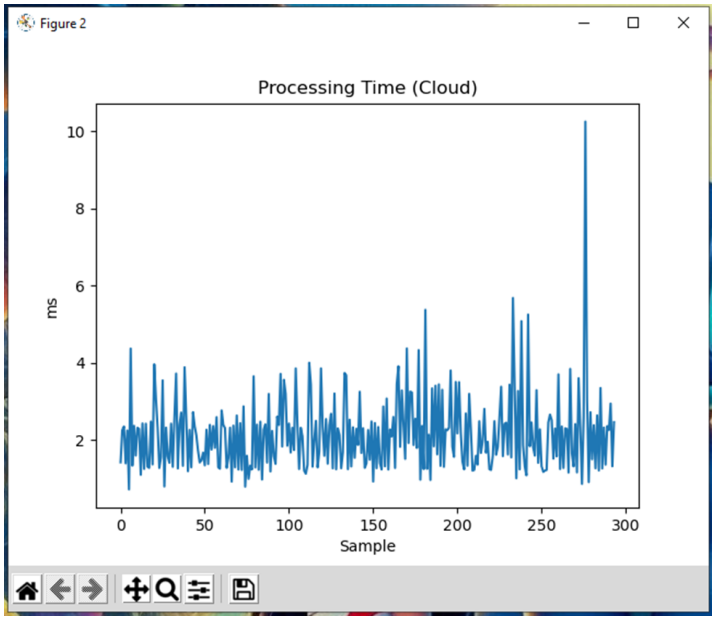

Figure 5: Processing Time pada Cloud Filtering

Berdasarkan Gambar processing time, waktu pemrosesan berada pada kisaran yang relatif kecil, dengan rata-rata kurang dari 1 ms per sampel. Hal ini menunjukkan bahwa Raspberry Pi memiliki kemampuan komputasi yang cukup untuk menjalankan Particle Filter secara real-time.

Secara keseluruhan, pendekatan cloud filtering mampu meningkatkan kualitas sinyal secara efektif, namun memiliki ketergantungan terhadap jaringan karena data mentah harus dikirim terlebih dahulu sebelum diproses. Pendekatan ini sesuai dengan konsep cloud computing yang memanfaatkan sumber daya komputasi terpusat untuk pengolahan data [5].

3.2 Hasil Pengujian Skenario B

Figure 6: Tampilan pada Arduino IDE saat running skenario B

Pada skenario B, proses filtering dilakukan langsung pada ESP32 sebelum data dikirim ke Raspberry Pi. Hal ini terlihat dari tampilan Arduino IDE yang menunjukkan nilai hasil filtering dihasilkan secara langsung bersamaan dengan data mentah.

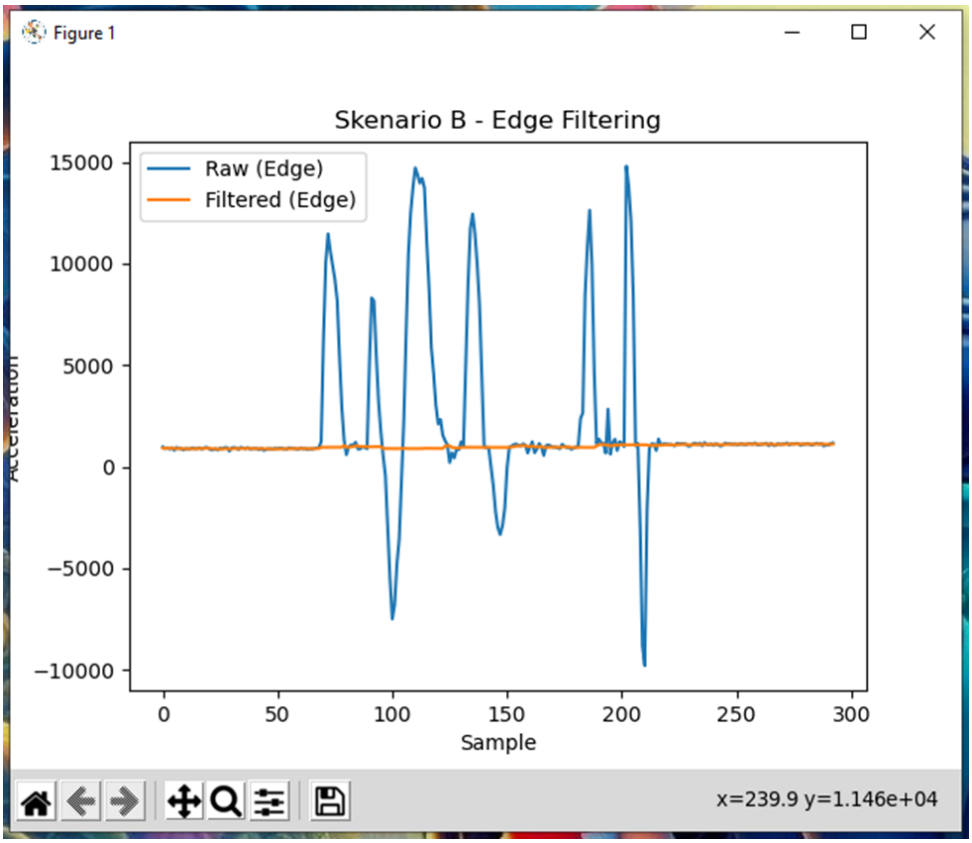

Figure 7: Edge Filtering

Hasil edge filtering menunjukkan bahwa sinyal mentah dengan fluktuasi tinggi dan outlier dapat direduksi secara signifikan. Sinyal hasil filtering menjadi lebih stabil dan tidak mengikuti lonjakan ekstrem, yang menunjukkan kemampuan Particle Filter dalam mempertahankan estimasi yang representatif terhadap kondisi sebenarnya.

Dibandingkan dengan skenario cloud filtering, hasil pada skenario edge menunjukkan tingkat kestabilan yang lebih baik. Hal ini disebabkan karena noise telah direduksi pada sumber data sebelum dikirim melalui jaringan. Selain itu, pendekatan ini juga meningkatkan efisiensi komunikasi karena data yang dikirim telah mengalami proses filtering.

Pendekatan ini sejalan dengan prinsip edge computing, di mana pemrosesan dilakukan lebih dekat dengan sumber data untuk mengurangi latency dan meningkatkan efisiensi sistem [2].

3.3 Pembahasan

Figure 8: Perbandingan Skenario A dan Skenario B

Berdasarkan hasil pengujian, terdapat perbedaan karakteristik yang jelas antara pendekatan cloud filtering dan edge filtering. Pada skenario A, sistem memiliki fleksibilitas komputasi yang lebih tinggi karena proses filtering dilakukan pada server. Namun, pendekatan ini memiliki ketergantungan terhadap jaringan dan potensi latency akibat transmisi data.

Sebaliknya, pada skenario B, filtering dilakukan langsung pada ESP32 sehingga menghasilkan respon yang lebih cepat dan sinyal yang lebih stabil. Hal ini menunjukkan bahwa edge filtering lebih efektif dalam meningkatkan efisiensi sistem dan kualitas data yang dikirim.

Perbedaan ini menunjukkan adanya trade-off antara fleksibilitas komputasi dan efisiensi sistem. Cloud computing memberikan kemampuan pemrosesan yang lebih kompleks, sedangkan edge computing memberikan keunggulan dalam hal real-time processing dan efisiensi komunikasi. Temuan ini sejalan dengan penelitian sebelumnya mengenai peran edge computing dalam sistem IoT modern [2].

Secara keseluruhan, edge filtering menunjukkan performa yang lebih baik dalam hal stabilitas sinyal dan efisiensi sistem, sedangkan cloud filtering tetap relevan untuk aplikasi yang membutuhkan komputasi yang lebih kompleks.

4. Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian sistem yang telah dilakukan, dapat disimpulkan bahwa metode Particle Filter berhasil diimplementasikan untuk mereduksi noise pada data sensor MPU6050 dalam sistem berbasis Internet of Things (IoT). Hasil pengujian menunjukkan bahwa algoritma Particle Filter mampu meningkatkan kualitas sinyal dengan menghasilkan data yang lebih stabil dibandingkan dengan data mentah.

Pada skenario cloud filtering, proses filtering yang dilakukan pada Raspberry Pi mampu mereduksi noise secara efektif dengan waktu pemrosesan yang relatif kecil, sehingga sistem dapat bekerja secara real-time. Namun, pendekatan ini memiliki ketergantungan terhadap jaringan karena data mentah harus dikirim terlebih dahulu sebelum diproses.

Sementara itu, pada skenario edge filtering, proses filtering yang dilakukan langsung pada ESP32 menghasilkan sinyal yang lebih stabil dan memiliki tingkat noise yang lebih rendah. Selain itu, pendekatan ini juga

mampu meningkatkan efisiensi sistem dengan mengurangi penggunaan bandwidth serta mempercepat respon sistem.

Secara keseluruhan, dapat disimpulkan bahwa kedua pendekatan memiliki keunggulan masing-masing. Cloud filtering lebih unggul dalam fleksibilitas komputasi, sedangkan edge filtering lebih efektif dalam meningkatkan stabilitas sinyal dan efisiensi sistem. Oleh karena itu, pemilihan metode filtering perlu disesuaikan dengan kebutuhan aplikasi yang digunakan.

5. Tantangan

1. **Komunikasi Wireless Menggunakan UDP**
   Tantangan utama yang dihadapi adalah memastikan komunikasi data antara ESP32 dan Raspberry Pi dapat berjalan dengan baik melalui protokol UDP. Proses ini memerlukan konfigurasi jaringan yang tepat, termasuk penggunaan alamat IP yang sesuai dan kestabilan koneksi WiFi. Selain itu, karena UDP tidak memiliki mekanisme pengecekan ulang data, diperlukan pengujian berulang untuk memastikan data dapat diterima dengan konsisten.

2. **Implementasi Particle Filter pada ESP32**
   Keterbatasan sumber daya pada ESP32, terutama dalam hal memori dan kemampuan komputasi, menjadi tantangan dalam mengimplementasikan Particle Filter. Diperlukan penyesuaian parameter seperti jumlah partikel dan nilai noise agar algoritma tetap dapat berjalan secara efisien tanpa mengurangi kualitas hasil filtering.

3. **Sinkronisasi dan Konsistensi Data**
   Dalam melakukan perbandingan antara skenario cloud filtering dan edge filtering, terdapat tantangan dalam menjaga konsistensi data yang digunakan. Perbedaan waktu pengambilan data dan kondisi gerakan sensor dapat mempengaruhi hasil analisis.

4. **Analisis dan Interpretasi Grafik**
   Proses analisis data memerlukan pemahaman yang baik dalam membaca grafik hasil filtering. Tantangan yang dihadapi adalah membedakan antara noise, outlier, dan pola sinyal yang sebenarnya, sehingga interpretasi hasil dapat dilakukan secara akurat.

6. New Skills Learned

1. **Implementasi Sistem IoT Berbasis ESP32**
   Penelitian ini memberikan pemahaman dalam merancang dan mengimplementasikan sistem IoT menggunakan ESP32 sebagai perangkat akuisisi data dan komunikasi wireless.

2. **Pengolahan Data Sensor Menggunakan Particle Filter**
   Penulis memperoleh pengalaman dalam mengimplementasikan algoritma Particle Filter untuk mereduksi noise pada data sensor, termasuk memahami tahapan prediksi, update, dan resampling.

3. **Komunikasi Data Menggunakan UDP**
   Kemampuan dalam membangun komunikasi data real-time menggunakan protokol UDP antara ESP32 dan Raspberry Pi menjadi salah satu keterampilan yang diperoleh.

4. **Pengolahan dan Visualisasi Data Menggunakan Python**
   Penulis memperoleh pengalaman dalam menggunakan Python, khususnya library NumPy dan Matplotlib, untuk melakukan pengolahan data numerik dan visualisasi grafik.

5. **Analisis Sistem Edge dan Cloud Computing**
   Penelitian ini memberikan pemahaman mengenai perbedaan karakteristik antara edge computing dan cloud computing, serta dampaknya terhadap performa sistem.

References

[1] M. Arulampalam, S. Maskell, N. Gordon, and T. Clapp, "A tutorial on particle filters for online nonlinear/non-gaussian bayesian tracking," *IEEE Transactions on Signal Processing*, vol. 50, no. 2, pp. 174–188, 2002.

[2] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.

[3] Espressif Systems, *ESP32 Technical Reference Manual*, 2020.

[4] InvenSense, *MPU-6000 and MPU-6050 Product Specification*, 2013.

[5] M. Satyanarayanan, "The emergence of edge computing," *Computer*, vol. 50, no. 1, pp. 30–39, 2017.